ÜBERBLICK

Projektüberblick

Was das Projekt ist, wie es aufgebaut ist und wie die Dokumente zusammenhängen

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	`C8205-Interpreter-Projektueberblick.md`	0.03-005
Stand	13.10.2026	
Umfang	8 Seiten, A4 hoch	
Erzeugt mit	`erzeuge_projektueberblick_pdf.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – Projektüberblick

Quelle: C8205-Interpreter-Projektueberblick.md · im Wortlaut der Quelle

Inhaltsverzeichnis

1. Ziel und Einordnung . . . 3

2. Zwei-Wege-Architektur: Python-Interpreter und Claude-Interpreter . . . 3

3. Teilprojekt Assembler . . . 4

3a. Teilprojekt BIS . . . 4

4. Virtuelle Hardware im Überblick . . . 5

5. Testkonzept: Trommeldatei, Tastenfolge, Testmarken, automatische Prüfung . . . 5

6. Dokumentenlandkarte (.md-Dateien und Code) . . . 6

7. PDF-Dokumente im Überblick . . . 8

8. Stand und offene Punkte . . . 8

C 8205 – Projektüberblick

Quelle: C8205-Interpreter-Projektueberblick.md · im Wortlaut der Quelle

Version: 0.03-005

Dieses Dokument richtet sich an Menschen, die das Projekt nicht im Detail verfolgt haben, und beschreibt zusammenhängend, was hier entsteht und wie die Einzeldokumente zueinander stehen. Es ersetzt keines der Fachdokumente, sondern führt in sie ein und verweist auf sie.

1. Ziel und Einordnung

Ziel des Projekts ist die **Simulation der Befehlsverarbeitung** des Cellatron C 8205, eines 1968 in der DDR gebauten Kleinrechenautomaten. Der C 8205 ist ein Einadreßautomat mit vergleichsweise geringem technischen Aufwand (rund 400 Transistoren, 2000 Dioden in der Zentraleinheit) und für einen Kleinrechner hoher Operationsgeschwindigkeit (bis zu rund 2000 verdrahtete Operationen je Sekunde bei zeitoptimaler Programmierung). Anstelle eines RAM-Speichers verwendet er eine Magnettrommel: Zugriffe auf einzelne Speicherzellen sind dadurch nicht beliebig schnell möglich, sondern hängen davon ab, wann die gewünschte Stelle der Trommel unter dem Lese-/Schreibkopf ansteht. Bedient wird der Rechner nicht über einen Bildschirm, sondern über ein Bedienpult mit Tasten, Schaltern und Lampen; Programme und Daten werden über Lochstreifen ein- und ausgegeben.

Die Simulation rekonstruiert diesen Rechner in zwei Schritten: Zunächst wird die Elektronik des Befehlsablaufs (die "vier Phasen" der Befehlsabarbeitung, phasenweise als Knotenautomat mit rund 80 Knoten beschrieben) aus Originalunterlagen so genau wie möglich als Regelwerk rekonstruiert. Darauf aufbauend wird dieses Regelwerk **interpretativ** ausgeführt — wortlautgetreu, ohne stillschweigende Vereinfachung: Ist eine Stelle des Regelwerks unklar, wird das als offener Punkt gemeldet statt eigenmächtig "repariert" ("laut scheitern"-Prinzip). Alle Zeitabhängigkeiten der realen Hardware (Wortzeit, Wartezeit der Trommel) werden für die Simulation auf 0 gesetzt: Die zeitliche Reihenfolge der Ereignisse bleibt erhalten, es entstehen aber keine echten Wartezeiten.

Das Gesamtprojekt gliedert sich dafür in drei Teilprojekte mit unterschiedlichem Zweck: Der **Interpreter** (Python, Wurzel des Projektverzeichnisses; Abschnitt 2) führt Maschinencode gegen das rekonstruierte Regelwerk aus, der **Assembler** (C, Unterverzeichnis `Assembler/`; Abschnitt 3) übersetzt lesbaren Assembler-Quelltext in genau diesen Maschinencode, und das **Bausteininterpretationssystem BIS** (Abschnitt 3a) ist das Programmsystem, das auf dem C 8205 selbst läuft. Interpreter und Assembler sind eigenständige Werkzeuge, die über den von ihnen erzeugten bzw. verarbeiteten Objektcode zusammenhängen, aber unabhängig voneinander entwickelt werden. Das BIS ist kein Werkzeug, sondern Anwendungs-Software für den C 8205; als Teilprojekt steht es gleichberechtigt neben den beiden anderen.

2. Zwei-Wege-Architektur: Python-Interpreter und Claude-Interpreter

Ein und dasselbe Testformat (Abschnitt 5) kann auf zwei unabhängigen Wegen ausgeführt werden, die absichtlich **auf gleiche Art und Weise** arbeiten — derselbe Knotengraph, dieselbe Testdatei, dasselbe Protokollformat:

- **Python-Interpreter** (`c8205_interp.py`): führt den Knotengraphen deterministisch und automatisiert aus. Schnell und reproduzierbar, aber ohne Rückfragemöglichkeit bei unklaren Regelwerk-Stellen. Er liest den Knotengraphen und die Variablentabelle T2 zur Laufzeit direkt aus den Regelwerk-Dateien (nicht fest einprogrammiert); nur die Wirkung jeder einzelnen ANWEISUNG ist von Hand als Python-Funktion hinterlegt. Fehlt zu einer ANWEISUNG eine Funktion, meldet der Interpreter das als Stub statt zu raten.
- **Claude-Interpreter**: Eine KI erhält das komplette Regelwerk als Kontext sowie dieselbe Testdatei und arbeitet den Knotengraphen manuell, Knoten für Knoten, ab. Langsamer und nicht deterministisch reproduzierbar (abhängig von Modell und Sitzung), erlaubt dafür aber Rückfragen und Erläuterungen während der Abarbeitung.

Rollenverteilung: Der Python-Interpreter übernimmt die laufenden Testläufe. Der Claude-Interpreter dient als unabhängige Gegenprobe — weichen die Ergebnisse beider Wege bei einem Test voneinander ab, liegt ein Fehler in einem der beiden vor (oder eine tatsächlich unklare Stelle im Regelwerk) — und kommt gezielt bei Rückfragen sowie beim Durchspielen neuer oder geänderter Befehle zum Einsatz, bevor diese in `c8205_interp.py` übernommen werden. Diese Aufgabenteilung gilt modellneutral für jede ausführende KI, nicht nur für Claude.

Wie ein Test auf beiden Wegen gestartet wird, welche Dateien dem Claude-Interpreter hochzuladen sind und wie die Startanweisung aussehen kann, steht in `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 1b.

3. Teilprojekt Assembler

Der Assembler `c82ass` (Cross-Assembler für C8205/C8205Z, Version 0.2, in C geschrieben, läuft unter Linux und Windows 10) ist ein eigenständiges Werkzeug, kein Interpreter oder Simulator: Er übersetzt lesbaren Assembler-Quelltext (`.asm`) in ladefähigen Objektcode (K1K2-Textformat) für das "Kleine Eingabeprogramm" des C 8205, denselben Maschinencode, den der Interpreter (Abschnitt 2) als Trommeldatei ausführt. Aufruf, Sprachdefinition, Anweisungen, Makros und Beispiel stehen in der Assemblerdokumentation (`Assembler/Dokumentation/`, die `.odt` ist maßgeblich); hier nur das Wesentliche:

- **Vier Pässe:** Pass 0 löst die Makros auf (`c82ass.makros`), Pass 1 ermittelt die Literaltabelle, Pass 2 die Symboltabelle (beide im `.log`, die Symbole zusätzlich in der `.sym`), Pass 3 erzeugt die Kennzeichenbefehle im Objektfile (`.obj`) sowie `.mqo`, `.dmp` und `.test`.
- **Sprache:** Maschinenbefehle (3 Oktalziffern) gemischt mit Assembleranweisungen: Datendefinition (`db`/`dbn` Bereich frei/auf Null, `dw` Wert, `da` Adresse, `dt`/`dto`/`dtm` Text als R300-Zeichenfolge, `dz` Sonderzeichen), Adressdefinition (`equ` relativ, `abs` absolut) und Programmrahmen (`start`/`end`). Adresssymbole (bis 32 Zeichen, mit Doppelpunkt abgeschlossen) und Literale (`#Werteausdruck` bzw. `@Adressausdruck`, vom Assembler automatisch auf einem eigenen Speicherplatz abgelegt) ergänzen die Sprache.
- **Makro-Präprozessor:** `MACRO`/`MEND`-Blöcke mit Parameterersetzung (`$1`, `$2`, …), bedingter Codegenerierung (`$IF`/`$ELIF`/`$ENDIF` mit bis zu vier über `|`/`&` verknüpften Vergleichsblöcken) und generischen, mehrfach verwendbaren Labels über einen `%`-Platzhalter, der bei jedem Aufruf durch eine fortlaufende Nummer ersetzt wird.
- **Erzeugte Dateien:** Wichtig sind `.obj` (Objektcode) und `.test` (Trommeldatei für den Interpreter, mit Platzhaltern für Testmarken und Tastenfolge); `.log`, `.mqo`, `.sym` und `.dmp` sind Tabellen, Listing und Hilfsdateien. Die vollständige Liste steht in `Projektstruktur.md`, Abschnitt 3.
- **Brücke zum Interpreter:** Die zusätzlich erzeugte `.test`-Datei enthält die vollständige Trommeldatei im Testformat des Interpreters (`Adresse: Wert`, oktal, mit der jeweiligen Quelltextzeile als Kommentar) sowie Platzhalter-Definitionen für Testmarken und Tastenfolge und ggf. den Leerbefehl `164`. Diese Platzhalter müssen für den jeweiligen Fall ergänzt bzw. erweitert werden (Debug-Nummern und Texte der Testmarken `#DOK-xxxx`, `#SPEI-xxxx`, `#PROT STUFE=n`, bei Bedarf `#AC=`/`#BZ=`, Tastenfolge z. B. `H G R`bei Bedarf `#AC=`/`#BZ=`/`#INTERRUPT`, Tastenfolge z. B. `H G R`bei Bedarf `#AC=`/`#BZ=`/`#INTERRUPT`, Tastenfolge z. B. `H G R` oder das Makro `#STARTE n`, optional Prüfungen). Danach wird die Datei als Testauftrag `Tests/asm_NNNN_<Programmname>.txt` abgelegt (Namensschema: `Projektstruktur.md`, Abschnitt 2). Der Code selbst stammt unverändert aus dem Assemblerlauf (`Durchführung_eines_Tests.md`, Abschnitt 2a).

3a. Teilprojekt BIS

Das Bausteininterpretationssystem (BIS; Herausgeber VEB Kombinat Zentronik) ist ein **offenes Unterprogrammsystem** für den C 8205 und das dritte Teilprojekt. Anders als Interpreter und Assembler ist es kein Werkzeug auf dem Entwicklungsrechner, sondern Software, die auf dem C 8205 selbst abläuft. Der Rechner kennt in Hardware nur die Eingabe **eines Zeichens**; alles Weitere (Worte, Adressen und Dezimalzahlen einlesen, Multiplikation und Division, Gleitkomma, Ausgabe) leisten Bausteine des BIS. Es enthält:

- **Eingabebausteine, Rechenbausteine, Ausgabebausteine und Serviceprogramme** (vier Gruppen), alle zeitoptimal aufgebaut, je Baustein 67_8 bis 377_8 Speicherplätze, unter festen Adressen erreichbar,
- das **Kleine Eingabeprogramm (KEP)** auf Spur 0 (Adressen 0000_8 bis 0037_8), das nur die Zeichen 0, 1, Punkt, Doppelpunkt und Zwischenraum verarbeitet und mit dem das Minimalprogramm geladen wird,
- das **Minimalprogramm**, den Grundbaustein, der Befehle, Adressen und Konstanten über die Eingabekennzeichen K_1 und K_2 einliest.

Stand der Erfassung: Dokumentiert sind bisher die allgemeine Beschreibung, das KEP und das Minimalprogramm; weitere Bausteine sind noch nicht erfasst (`C8205-Interpreter-BIS.md`).

Beziehung zu den anderen Teilprojekten:

- Die Gleitkomma-, Festkomma- und Tetradendarstellung der Zahlen sowie ihre externe Darstellung sind Konventionen des BIS (`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`), nicht der Rechner-Hardware.
- Der Objektcode des Assemblers (K1K2-Format, Abschnitt 3) ist für das KEP bestimmt, also für den ersten Baustein des BIS.

- Interpreter und Assembler kennen das BIS nicht: Es ist kein Teil der simulierten Hardware. Anmerkung (Folgerung, nicht aus der Quelle): Bausteine ließen sich als Trommeldatei eines Tests im Interpreter ausführen.

Einzelheiten stehen nicht hier, sondern in `C8205-Interpreter-BIS.md`. Das Format der Objektdatei, die das KEP liest, und der Weg vom Vorgabebild zum Testauftrag stehen in `C8205-Interpreter-Objektdatei_Listing.md` (ebenfalls im Ordner `Bausteininterpretiersystem BIS/markdown-dateien/`).

4. Virtuelle Hardware im Überblick

Der reale C 8205 (ECHT) besteht aus Stromversorgung, Zentraleinheit (Rechenwerk und Leitwerk) und Peripherie: Magnettrommelspeicher, Bedienpult mit Tastatur/Lampenfeld/Bahnsperrenschaltern, zwei Lochbandlesern, einem Lochbandstanzer und einem Schreibwerk für alphanumerische Ein-/Ausgabe. Für die Simulation wird davon **virtualisiert**, was ein Testlauf am Bildschirm bzw. in Dateien braucht — mit denselben Kanal- und Adresszuordnungen wie beim Original, aber ohne mechanische Bauteile:

- **Bedienpult:** Tasten (E, G, M, Ü, ST u. a.) und Schalter wirken als Tastenfolge auf Register und Flags, statt tatsächlich gedrückt zu werden; Lampen werden durch die protokollierten Flag-Werte ersetzt.
- **Magnettrommel:** statt der physischen Trommel eine persistente Textdatei mit allen 4096 Zellen (`virtuelle_Geraete/Trommel/trommel.txt`, Adresse:Wert je Zeile, oktal). Sie behält ihren Inhalt über Tests hinweg. Ein Test gibt die Zellen, die er braucht, vorab als Textdatei (Adresse:Wert je Zeile, oktal) mit; sie werden auf die Trommel geschrieben und überlagern deren Inhalt.
- **Lochbandleser (2, Kanäle K1/K2):** je Leser eine virtuelle Streifendatei; ist keine angegeben und fordert das Programm dennoch eine Eingabe an, bleibt der Lauf mit gesetztem Flag E stehen wie beim Original ohne eingelegten Streifen — kein Fehler.
- **Lochbandstanzer (Kanal K1):** Ausgaben werden zeilenweise binär (Spuren w v u p 8 4 2 1) in eine Ausgabedatei geschrieben, statt real gestanzt zu werden.
- **Schreibwerk (Ausgabe über Kanal K2):** zeichenweise Textausgabe, mit WRZL (Wagenrücklauf Zeilenvorschub) als Zeilenendekennung, statt auf eine Schreibmaschine: in die Datei `sw-ausgabe.txt`, wenn dem Interpreter eine Datei zugeordnet ist (`--sw-ausgabe`), sonst auf das Terminal.
- **Tastatur und Schreibwerk-Eingabe (Kanal K0):** Das Zeichen kommt im Standardfall von der Tastatur des Host-Systems, mit `--k0 datei=PFAD` aus einer Datei.

Hilfsprogramme für die Lochstreifen: Zwei eigenständige Python-Programme arbeiten mit denselben virtuellen Lochstreifen wie der Interpreter. `c8205_lochstreifen_anzeige.py` schreibt einen Lochstreifen (etwa die Ausgabe von `--stanzer1`) als lesbaren Text in einem Fenster aus; `c8205_lochstreifen_eingabe.py` erzeugt aus Tastatureingaben einen Lochstreifen, den man dem Interpreter mit `--leser1`/`--leser2` übergeben kann. Beide gehören zum Interpreter-Teilprojekt, werden vom Interpreter aber nicht aufgerufen (Doku: `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`).

Jede virtuelle Datei wird nur angelegt, wenn der jeweilige Test das Gerät tatsächlich benutzt, und am Testende zum Download bereitgestellt. Ausnahme ist die Trommeldatei: Sie muss bei jedem Lauf vorhanden sein und wird vom Interpreter nie angelegt (Hilfsskript `erzeuge_leere_trommel.py`).

5. Testkonzept: Trommeldatei, Tastenfolge, Testmarken, automatische Prüfung

Jeder simulierte "Rechnerlauf" ist als **Test** definiert, unabhängig davon, ob er anschließend vom Python- oder vom Claude-Interpreter ausgeführt wird. Ein Test besteht aus:

- **Trommeldatei** (Abschnitt `Trommeldatei:` der Testdatei): der für den Test relevante Ausschnitt des Speichers, zeilenweise als `Adresse: Wert` (beide oktal), mit Kommentaren (`#`) und Leerzeilen. Er wird vor dem Test auf die persistente Trommel geschrieben; nicht genannte Zellen behalten ihren bisherigen Inhalt (`Durchführung_eines_Tests.md`, Abschnitt 2c).
- **Tastenfolge:** definiert den Ausgangszustand von Registern und Flags vor Beginn der Befehlsschleife — welche Tasten/Schalter in welcher Reihenfolge betätigt werden (z. B. `H G R` für einen regulären Start), einschließlich der jeweiligen Wirkung auf Register und Flags.
- **Testmarken** für die Auswertung, ohne Entsprechung in der realen Elektronik:
  - `#DOK-xxxx <Text>` löst bei Erreichen des Leerbefehls 164 an Adresse xxxx eine Diagnoseausgabe aus (Text, Register BZ/AC, Flags FE/FÜ/BS/US/E/A).

  - `#SPEI-xxxx ssss-eeee` gibt zusätzlich einen Speicherbereich aus.
  - `#SPUR-xxxx aaaa` gibt zusätzlich eine ganze Trommelspur in physischer Zugriffsreihenfolge aus.
  - `#PROT STUFE=n` (0/1/2) steuert die Protokolltiefe, bis hin zu einer vollständigen, Knoten-für-Knoten protokollierten Protokolldatei bei STUFE=2.
  - `#AC=` (11 Oktalziffern) und `#BZ=` (4 Oktalziffern) geben optional die Anfangswerte von Akkumulator und Befehlszähler vor der ersten Taste vor (Ergänzung 08.10.2026); ohne sie starten beide mit 0. Die Taste `G` wirkt danach wie bisher.
  - `#STARTE n` (Ergänzung 09.10.2026) steht für die Tastenfolge `H G M+ E … E M- R` (Ansprung des Speicherplatzes n, `Durchführung_eines_Tests.md`, Abschnitt 2e).
  - `#INTERRUPT H <Tasten>` (Ergänzung 08.10.2026) unterbricht den Programmlauf zwischen zwei Befehlszeilen mit einer weiteren Tastenfolge (`Durchführung_eines_Tests.md`, Abschnitt 2d).
- **Prüfungen** (optional, Abschnitt `Prüfungen:` am Ende der Testdatei): Sollwerte für Register, Flags und Speicherzellen sowie Dateiprüfungen für die Stanzer- und die Schreibwerk-Ausgabe; das Gesamtergebnis lautet *Test wahr* oder *Test falsch* (`Durchführung_eines_Tests.md`, Abschnitte 3a und 3b).

Automatische Prüfbarkeit: Weil derselbe Test mit demselben Format deterministisch (Python) oder als Gegenprobe (Claude) ausgeführt werden kann, lassen sich für jeden Test **Sollwerte** festhalten — Endzustand von Register/Flags, erwartete Speicherinhalte, erwartete Ausgaben. Damit wird jeder Testlauf zu einem Regressionstest: Ein neuer Lauf wird automatisch gegen die zuletzt bestätigten Sollwerte verglichen, ohne dass ein Mensch das Protokoll erneut Zeile für Zeile lesen muss. Voraussetzung ist ein definierter Trommelstand: Weil die Trommel Tests überdauert, gelten Sollwerte für einen Lauf auf leerer Trommel oder für einen Test, der alle benutzten Zellen selbst nennt (Abschnitt 2c der Testbeschreibung). Referenzläufe dieser Art sind bereits für mehrere Tests festgehalten (u. a. Tests `0004` bis `0021`, `s1-R1` bis `s4-spur`).

Ergänzend dazu prüft `c8205_befehlssweep.py` systematisch alle 512 möglichen Operationscodes (000–777 oktal) gegen den Knotengraphen: Für jeden Code wird ein Minimalprogramm gebaut und direkt über dieselbe Knotengraph-Engine wie `c8205_interp.py` ausgeführt. Das prüft nicht die numerische Richtigkeit jeder Formel — dafür bleiben die inhaltlichen Einzeltests zuständig —, sondern ob für jeden Code überhaupt eine ANWEISUNG-Implementierung vorhanden ist und der Automat sauber terminiert (kein Stub, kein Sackgassen-Knoten, keine erkennbare Endlosschleife): ein automatisierter Vollständigkeits-Regressionstest zusätzlich zu den inhaltlichen Einzeltests.

6. Dokumentenlandkarte (.md-Dateien und Code)

Datei	Rolle
`C8205-Einstieg_fuer_Neueinsteiger.md`	Orientierung für neue Leser des Gesamtprojekts (Interpreter, Assembler, BIS): was die Teile bedeuten, empfohlene Lesereihenfolge, was man zunächst nicht lesen muss, Zuordnung Frage → Dokument
`C8205-Interpreter-Überblick_und_Gesamtaufbau.md`	Wurzeldokument: Einordnung, Geräteaufbau, Rechen-/Leitwerk, die vier Phasen der Befehlsabarbeitung, Speicher-Timing und Leistungswerte
`C8205-Interpreter-Magnettrommel.md`	Speicherorganisation der Trommel (128 Spuren × 32 Sektoren), Interleaving/physische vs. logische Sektoradresse, Bahnsperrenschalter; Nachbildung im Interpreter als persistente Datei
`C8205-Interpreter-Bedienpult_mit_Schaltern.md`	Bedienelemente: Tasten, Schalter, Kanalanwahl, Wirkung auf Register/Flags
`C8205-Interpreter-Lochstreifenleser.md`	Hardwarebeschreibung Lochbandleser, -stanzer und Schreibwerk (Kanäle, Ablauf einer Zeichen-Ein-/Ausgabe)
`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`	Register (AC, BZ, AR, OR, BR), Flags, Maschinenwort- und Befehlswortaufbau (Bitfelder)
`C8205-Interpreter-Eingänge_der_Knoten.md`	Der Knotengraph selbst: alle Knoten (Konnektor/Phasenzuordnung/Anweisung/Test), Ein-/Ausgänge, Verzweigungen

C 8205 – Projektüberblick

Quelle: C8205-Interpreter-Projektueberblick.md · im Wortlaut der Quelle

Datei	Rolle
C8205-Interpreter-Beschreibung_der_Knoten.md	Variablentabelle T2: Bedeutung jeder ANWEISUNG-nnn, die im Knotengraph referenziert wird
C8205-Interpreter-Befehlsliste.md	Generische Befehlsbeschreibung nach B/A/Z sowie die vollständige, nach Oktalcode geordnete Einzelbefehlsübersicht (372 Befehle)
C8205-Interpreter-Kodetabellen.md	Zeichenkodetabelle: R300-Lochstreifenkode (Tabelle 1)
C8205-Interpreter-BIS.md	Teilprojekt BIS (Abschnitt 3a): Bausteininterpretationssystem, Kleines Eingabeprogramm (KEP) auf Spur 0, Minimalprogramm mit den Eingabekennzeichen K_1/K_2
Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-Objektdatei_Listing.md	Objektdatei-Ausdrucke der Beispielprogramme (Vorgabebilder): Spalten einer Befehlszeile, Relokationskennung K1K2, vom Vorgabebild zum Testauftrag, Beispiel Maximum von drei Zahlen; liegt im BIS-Ordner
C8205-Interpreter-Durchführung_eines_Tests.md	Das Testkonzept selbst (Abschnitt 5 dieses Dokuments): Trommeldatei, Tastenfolge, Testmarken, virtuelle Geräte, Protokollierung, Python-/Claude-Interpreter, Start eines Tests auf beiden Wegen (Abschnitt 1b)
C8205-Interpreter-Lochstreifen-Hilfsprogramme.md	Bedienung und Technik der beiden Lochstreifen-Hilfsprogramme (Anzeige, Eingabe): Format des virtuellen Lochstreifens, Zeichenlage, Irrung, Kopieren in die Zwischenablage, Installation unter Linux, Windows, macOS
C8205-Interpreter-Python_Interpreter.md	Bedienung von `c8205_interp.py`: Installation, Aufruf, Schalter, Fehlermeldungen, Beispiele; Vorgaben für die Bedienungsanleitung (PDF)
baz-wirkungstabelle.md	Cache bereits hergeleiteter Knotenpfade/Formeln je getestetem BAZ-Wert, um wiederholte Herleitung zu vermeiden
C8205-Projektstruktur.md	Wurzeldokument der Verzeichnisstruktur: welches Verzeichnis/welche Endung welchen Zweck hat, für Interpreter und Assembler
C8205-Versionierung.md	Wurzeldokument der Versionsnummern: Aufbau `y.xx-zzz`, welche Dateien eine tragen, wo sie steht und wann sie sich ändert
C8205-Interpreter-Projektueberblick.md	dieses Dokument: Ziel, Zwei-Wege-Architektur, Teilprojekte, virtuelle Hardware, Testkonzept, diese Dokumentenlandkarte, PDF-Übersicht, Stand
README.md, README.en.md	Einstiegsseiten für GitHub (deutsch, englisch) im Wurzelverzeichnis: fassen das Projekt auf einer Seite zusammen und verweisen auf die Dokumente; keine eigenen Regeln
C8205_Assembler_Dokumentation.md	Durchsuchbare Textableitung der maßgeblichen `C8205_Assembler_Dokumentation.odt`: Sprache, Anweisungen, Makro-Präprozessor, Beispiel
C8205-Interpreter-PDF.md	Gestaltungsrichtlinie für alle PDF-Ausgaben; Dateibenennung und Liste der erzeugten PDFs (Abschnitt 8)

Alle Dateien werden zwischen Sitzungen erneut hochgeladen, da sie sich auch außerhalb einzelner Gespräche weiterentwickeln.

Jede Datei trägt eine Versionsnummer (`Version: y.xx-zzz`). Wer eine Datei ändert, erhöht deren Build (`zzz`) um 1; Hauptversion und Unterversion werden nur auf ausdrückliche Aufforderung des Projektinhabers erhöht (`C8205-Versionierung.md`).

Code-Dateien (kein Regelwerk, sondern dessen Umsetzung bzw. Werkzeuge dafür):

Datei	Rolle
`c8205_interp.py`	Der Python-Interpreter selbst (Abschnitt 2): liest Knotengraph und T2 zur Laufzeit aus den .md-Dateien, führt Tests aus
`c8205_befehlssweep.py`	Vollständigkeitstest über alle 512 Operationscodes (Abschnitt 5)
`erzeuge_leere_trommel.py`	Erzeugt bzw. setzt die Trommeldatei `virtuelle_Geraete/Trommel/trommel.txt` auf eine leere Trommel zurück (alle 4096 Zellen 0)
`c8205_lochstreifen_anzeige.py`	Hilfsprogramm: schreibt einen virtuellen Lochstreifen als Klartext in einem Fenster aus (Abschnitt 4)
`c8205_lochstreifen_eingabe.py`	Hilfsprogramm: erzeugt einen virtuellen Lochstreifen aus Tastatureingaben (Abschnitt 4)
`Assembler/Sourcecode/` (C-Dateien)	C-Quellcode des Assemblers `c82ass` selbst (Abschnitt 3)

7. PDF-Dokumente im Überblick

Zusätzlich zu den .md-Quellen liegen gut lesbare, A4-taugliche PDF-Ausgaben vor (Ordner `PDF/`), gestaltet nach einheitlicher Richtlinie (`C8205-Interpreter-PDF.md`). Die PDFs sind fertige Dokumente und werden nicht automatisch aus den .md-Dateien nachgeführt; sie können daher einen älteren Stand haben als ihre Quelle (der Stand steht auf dem Deckblatt bzw. in der Kopfzeile). Dateibenennung und Quelldateien: PDF-Richtlinie, Abschnitt 8.

PDF	Inhalt
`C8205-Projektueberblick_A4.pdf`	dieses Dokument
`C8205-Rechnerbeschreibung_A4.pdf`	Aufbau, Register, Maschinenwort, Magnettrommel, Befehlszyklus und Befehlsarten des Rechners (redaktionell)
`C8205-Bedienpult_und_Peripherie_A4.pdf`	Tasten, Schalter, Lampen, Kanäle, Lochstreifenleser, -stanzer und Schreibwerk; Token der Tastenfolge (redaktionell)
`C8205-Bedienungsanleitung_Interpreter_A4.pdf`	Bedienung von `c8205_interp.py`: Aufruf, Schalter, Geräte (redaktionell)
`C8205-Testhandbuch_A4.pdf`	Kurzanleitung „Test starten" (Claude- und Python-Interpreter), Tests schreiben und Protokolle lesen (redaktionell)
`C8205-Durchfuehrung_eines_Tests_A4.pdf`	Das Testformat im Wortlaut der Quelle: Trommeldatei, Tastenfolge, Testmarken, Prüfungen, virtuelle Geräte, Protokoll
`C8205-Knotengraph_A4.pdf`	Der komplette Knotengraph als Diagramm, auf A4-Kacheln aufgeteilt
`C8205-Knotentabellen_A4.pdf`	Knotentabelle und Variablentabelle T2 im Wortlaut der Quellen
`C8205-Einzelbefehlsliste_A4.pdf`	Generische Befehlsbeschreibung und vollständige Einzelbefehlsübersicht
`C8205-Kodetabellen_A4.pdf`	Die Zeichenkodetabelle (Tabelle 1), Lochbild-Spalten als ●/○-Grafik
`C8205-Assemblerdokumentation_A4.pdf`	Assembler `c82ass`: Aufruf, Sprache, Anweisungen, Makros, Beispiel; liegt in `Assembler/Dokumentation/` (im Wortlaut der `.md`, die `.odt` bleibt maßgeblich)

8. Stand und offene Punkte